

Frosch Bottle Inspection

Model Training Record

RF-DETR Medium · Detection Training · Validation Metrics · ONNX · TensorRT

Training Evidence Record (A07-style)

Prepared against the xis.ai AI Departmental Handbook v1.0

TRAINING RECORD · Version 1.0 · August 2026

Field	Details
Project	Frosch Bottle Inspection
Department	AI Department, xis.ai
Document scope	Model training only (detection RF-DETR Medium)
Model	RF-DETR Medium
Dataset	frosch-bottle-5-ypv4gi · 1,046 annotated images
Training device	NVIDIA GeForce RTX 5060
Export forms	PyTorch checkpoint → ONNX → TensorRT
Primary status	Training complete; evaluation evidence recorded
Confidentiality	Internal use only

Scope. This document records the detection-model training run only. It does not cover segmentation training, OCR, geometry rules, tracking, or production release. It is training evidence, not a production approval.

Contents

- §1 Training Summary
- §2 Dataset Used for Training
- §3 Model and Training Configuration
- §4 Class Definitions and Order
- §5 Recorded Validation Metrics
- §6 Export — ONNX and TensorRT
- §7 Runtime Observation (TensorRT)
- §8 Limitations and Open Items

Document purpose. Record what was trained, on which data, with which settings, and what validation metrics were obtained. Numbers not measured (ONNX/CPU FPS, exact checkpoint SHA-256, formal sealed test-set protocol) are left explicit rather than invented.

§1 Training Summary

A single RF-DETR Medium detection model was trained for multi-class bottle inspection (bottle, label, capacity, damage, bump, scratch). Training used 100 epochs with no early stopping, learning rate 0.001, batch size 8, on an NVIDIA GeForce RTX 5060. The dataset contained 1,046 COCO-annotated images, split 80% train / 20% validation. No data augmentation was applied. The resulting checkpoint was exported to ONNX and converted to TensorRT for the live pipeline.

Item	Recorded value
Model family	RF-DETR Medium
Task	Object detection (multi-class)
Epochs	100
Early stopping	Not used
Learning rate	0.001
Batch size	8
Augmentation	None applied
Annotation format	COCO
Images	1,046 (annotated once)
Split	80% train / 20% validation
Device	NVIDIA GeForce RTX 5060
Input size (standard RF-DETR)	576 × 576

§2 Dataset Used for Training

Field	Recorded value
Dataset ID	frosch-bottle-5-ypv4gi
Total annotated images	1,046
Annotation rounds	One (single pass)
Annotation format	COCO
Train share	80%
Validation share	20%
Augmentation during training	None
Domain	Frosch bottles (100 / 300 / 500 ml)

Images were annotated once. The split is train/validation only; a separate sealed hold-out test set protocol is not recorded in this training document.

§3 Model and Training Configuration

3.1 Model

RF-DETR Medium was selected as the detection backbone. It is used in the operational pipeline as a TensorRT engine (output/rfdetr-medium.trt). Training lineage checkpoint path referenced in the pipeline: runs/frosch_medium/checkpoint_best_regular.pth.

3.2 Hyperparameters

Hyperparameter	Value
Epochs	100
Early stopping	Disabled (full 100 epochs run)
Learning rate	0.001
Batch size	8
Input resolution	576 × 576 (RF-DETR standard)
Augmentation	None
Framework context	RF-DETR / PyTorch training stack
Hardware	NVIDIA GeForce RTX 5060

3.3 Why these settings (short)

- 100 epochs without early stop — full schedule completed for a stable final checkpoint.
- LR 0.001 — standard starting rate used for this RF-DETR training run.
- Batch size 8 — fits RTX 5060 memory for the Medium model at 576×576.
- No augmentation — training used the annotated images as labelled, without geometric/colour transforms.

§4 Class Definitions and Order

Detection classes used in training and evaluation (order as in the per-class metrics chart):

#	Class	Role in inspection
1	bottle	Primary object; tracking and geometry
2	bump	Defect candidate
3	capacity	OCR region for 100 / 300 / 500
4	damage	Defect candidate
5	label	Centricity (H/V) measurement
6	scratch	Present in model vocabulary

Operational confidence thresholds used later in the live pipeline (not training hyperparams): bottle 0.70 · label 0.35 · capacity 0.35 · bump 0.50 · damage 0.30 · scratch 0.30.

§5 Recorded Validation Metrics

Per-class Precision, Recall and AP50 from the validation evaluation of the trained detector. Values read from the project per-class metrics chart (approximate to chart scale; use exact export numbers if a later A17 locks them).

Class	Precision	Recall	AP50
bottle	~99%	~100%	~91%
bump	~45%	~55%	~41%
capacity	~92%	~100%	~91%
damage	~36%	~92%	~84%
label	~99%	~100%	~91%
scratch	~33%	~21%	~20%

Reading the table. bottle, capacity and label are strong on precision/recall and AP50. damage shows high recall but lower precision (more false positives). bump and scratch are weaker overall — lower precision, recall and AP50 — which matches the higher miss/confusion rates seen in the $\text{IoU} \geq 0.50$ confusion matrix for those classes.

5.1 Confusion matrix context ($\text{IoU} \geq 0.50$)

A separate evaluation confusion matrix ($\text{IoU} \geq 0.50$) was recorded earlier for the same class set. Strong diagonals for bottle, capacity and label; higher misses for bump and scratch. That matrix is retained as complementary evaluation evidence and is not re-derived here.

§6 Export — ONNX and TensorRT

The trained checkpoint was exported to ONNX and then converted to a TensorRT engine for the live inspection pipeline.

Stage	Artifact / note
Training checkpoint (lineage)	runs/frosch_medium/checkpoint_best_regular.pth
ONNX export	Performed (intermediate portable graph)
TensorRT engine	output/rfdetr-medium.trt (operational path)
Runtime execution	Native TensorRT, dedicated CUDA stream
Input (runtime)	RGB frame staged via pinned host buffer → engine preprocess
Batch	Static batch 1 in the current runner

ONNX and pure CPU FPS were not measured for this record. Only TensorRT inference throughput on the project GPU path is available (see §7).

§7 Runtime Observation (TensorRT)

Metric	Scope	Sample value
Average inference FPS	Preprocess → H2D → TensorRT detection (+ paired seg in full pipe) → D2H	~74.66 FPS
Full pipeline FPS	Capture → detect → track → OCR → draw → save → display (bottle present)	~18.66 FPS

These figures are sample operational observations on the project setup with TensorRT. They are not a formal sealed A17 soak on a locked production image. ONNX Runtime / CPU throughput was not concluded and is left open.

§8 Limitations and Open Items

- No early-stopping curve or epoch-wise metric log is attached to this record.
- Checkpoint SHA-256 and full environment lock hash are not recorded here.
- Train/val split is 80/20; a separate sealed test set and leak check are not part of this training note.
- ONNX and CPU FPS not measured.
- scratch and bump remain the weakest classes on the recorded validation metrics.
- This document is training evidence only — not A13 release or production approval.

Status. *Training of RF-DETR Medium for Frosch detection is complete under the settings above. Validation metrics and TensorRT export are recorded. Further handbook artifacts (A02 objective, A06 sealed dataset version, A09 validation, A11 soak, A13 release) remain outside the scope of this training-only document.*